TECHNICAL REPORT

1. PROBLEM STATEMENT:

Despite advancements in automotive safety, human error remains the primary cause of traffic accidents, particularly in unpredictable urban environments involving Vulnerable Road Users (VRUs) like pedestrians. While fully autonomous (Level 5) vehicles are the ultimate goal, the current transitional phase requires robust Level 2+ Advanced Driver Assistance Systems (ADAS).

The primary problem addressed in this project is the development of a real-time, sensor-fused ADAS that operates symbiotically with a human driver. The system must process noisy environmental data (via radar), filter out irrelevant ground clutter, and execute high-priority safety interventions— such as Autonomous Emergency Braking (AEB) and Adaptive Cruise Control (ACC)— without overriding the driver's manual control unless absolutely necessary for collision avoidance. The challenge lies in creating an arbitration logic that perfectly balances driver autonomy with deterministic safety fallbacks in a simulated urban environment (CARLA).

2. SYSTEM OVERVIEW:

The developed ADAS architecture is a manual-first, safety-overlay system designed for the CARLA Simulator. It features a custom Human-Machine Interface (HMI) built with Pygame and utilizes a synchronized client-server communication model to ensure deterministic physics and sensor processing.

Key Subsystems:

Perception C Sensor Layer: Utilizes a forward-facing radar with a customized Field of View (35° horizontal, 15° vertical) to detect vehicles and pedestrians while filtering out ground hits. A 3rd-person RGB camera provides visual feedback to the HMI.

Arbitration C Logic Layer: The "brain" of the system. It continuously calculates the Time-to-Collision (TTC) and evaluates driver inputs (W,A,S,D) against environmental hazards.

Active Safety Features:

AEB (Autonomous Emergency Braking): A multi-stage collision avoidance system with a 6.0-meter panic threshold and dynamic TTC evaluation.

ACC (Adaptive Cruise Control): A speed-governed cruise system that maintains a dynamic 2.2-second time gap from leading obstacles using a PID controller.

LKA (Lane Keeping Assist): A proportional lateral controller that nudges the steering to keep the ego vehicle centered, provided the driver is not actively initiating a turn.

3. METHODOLOGY:

The methodology prioritizes system stability and deterministic behavior. In a highly complex simulator like CARLA, asynchronous sensor processing can lead to "ghost braking" or control deadlock.

2.1 Synchronous Simulation State

To resolve simulator instability, the CARLA World and Traffic Manager were locked into a Synchronous Mode with a fixed time-step (fixed_delta_seconds = 0.05). This ensures that the sensor point-clouds, the Python arbitration logic, and the physics actuation all execute in the exact same frame, eliminating latency-induced crashes.

2.2 Sensor Noise Mitigation

Raw radar data is inherently noisy. To prevent the AEB from triggering on false positives (like the asphalt or map curbs), a strict geometric filter was applied. The radar parsing algorithm drops any points where the altitude exceeds 0.05 radians or the depth is less than 0.5 meters.

2.3 Mutual Exclusivity in Actuation

A core principle of the methodology is hardware safety simulation. To prevent the simulated actuators from "fighting" (applying brake and throttle simultaneously), a mutual exclusivity filter was implemented. If the smoothed brake signal exceeds 10%, the throttle signal is aggressively zeroed out.

4. ALGORITHMS & CALCULATIONS:

2.4 Time-to-Collision (TTC) G Dynamic Spacing

The system evaluates collision risk using a stabilized Time-to-Collision (TTC) calculation. Since radar relative velocity (v_rel) may fluctuate due to noise or stationary targets, the system computes a conservative closing speed using both relative velocity and ego vehicle velocity.

Closing speed:

v_closing = max(v_rel , 0.5 * v_ego)

This ensures that even if the detected object is stationary, the system accounts for the ego vehicle’s motion.

TTC is then calculated as:

TTC = d_rel / v_closing Where:

- d_rel = relative distance to object (meters)
- v_closing = stabilized closing velocity (m/s)

To prevent unstable or infinite values when the vehicle is nearly stationary: TTC is only computed if v_closing > 0.5 m/s

TTC Thresholds

Forward Collision Warning (FCW):
TTC < 3.5 seconds

Autonomous Emergency Braking (AEB):
TTC < 1.8 seconds

Dynamic Safe Distance for ACC

Adaptive Cruise Control does not use a fixed following distance. Instead, it follows the 2.2-second time-gap rule:

D_safe = max(v_ego * 2.2 , 12.0) Where:

- v_ego is in m/s
- 2.2 seconds represents a safe reaction buffer
- 12.0 meters is the minimum enforced spacing

This ensures sufficient stopping margin at all urban speeds.

2.5 One-Dimensional Kalman Filter (State Estimation)

To reduce radar measurement noise, a 1D Kalman Filter is applied separately to distance and velocity.

Let:

- x_t = estimated state at time t
- p_t = estimation uncertainty
- q = process noise
- r = measurement noise
- z_t = measurement at time t

Prediction Step p_t

$= p_{(t-1)} + q$ Kalman

Gain

$K_t = p_t / (p_t + r)$

Update Step

$x_t = x_{(t-1)} + K_t * (z_t - x_{(t-}$

$1)) p_t = (1 - K_t) * p_t$

This recursive filtering reduces sudden measurement spikes and stabilizes TTC calculations.

2.6 PID Control for Adaptive Cruise Control (ACC)

To prevent jerky acceleration and braking, ACC throttle control uses a PID controller.

Error term:

$e(t) = v_{target}(t) - v_{ego}(t)$

Control output:

$u(t) = Kp * e(t)$
$+ Ki *$ integral of $e(t)$ dt
$+ Kd *$ derivative of $e(t)$

Tuned gains for urban operation:

Kp = 0.12
Ki = 0.01
Kd = 0.03

This tuning:

- Reduces overshoot
- Prevents oscillations
- Ensures smooth speed convergence

2.7 Low-Pass Actuator Smoothing (Exponential Moving Average)

To avoid abrupt transitions between manual and ADAS control, an exponential smoothing filter is applied to final actuator commands.

Control_t = alpha * Target_t + (1 - alpha) * Control_(t-1) Where:

- alpha = 0.4
- Target_t = new control command
- Control_(t-1) = previous control value

This smoothing:

- Prevents jerky inputs
- Simulates mechanical actuator latency
- Improves driver comfort perception

5. ARCHITECTURE:

The software architecture follows a decoupled, modular design, ensuring that sensor processing does not bottleneck control actuation:

* Input C Environment Layer: The CARLA server generates the HD Map, weather, and AI actors. The Pygame event listener captures manual human inputs (W, A, S, D).

* Perception Layer: The _process_radar callback ingests arrays of carla.RadarDetection, filters them geometrically, and feeds them into the Kalman Filter objects.

* Arbitration Layer (The Decision Engine): This is the core logical router. It compares manual human inputs against the filtered sensor data and TTC thresholds.

* Level 1 (Absolute Priority): AEB. If TTC < 1.8, manual throttle is ignored, and full brakes are applied.

* Level 2 (Active Assist): ACC. If the driver is holding throttle but approaching a vehicle too fast, throttle is reduced.

* Level 3 (Passive Assist): LKA. Applies additive steering to keep the car centered.

* Actuation C HMI Layer: The final smoothed signals are packaged into a carla.VehicleControl object and sent to the server. Simultaneously, telemetry is routed to the Pygame HMI for rendering.

6. RESULTS & OBSERVATIONS:

The implemented system was tested under heavy urban traffic conditions in CARLA's Town10HD map.

* Pedestrian Safety (VRU Protection): The most significant result was the system's ability to detect pedestrians crossing unpredictably. The widened radar FOV successfully captured pedestrians stepping off curbs. The inclusion of the 6.0-meter absolute panic threshold ensured AEB activation even when pedestrians were stationary, preventing 100% of low-speed urban collisions during testing.

* Traffic ACC Performance: The PID-controlled ACC successfully tracked lead vehicles. The dynamic distance calculation (D_{safe}) prevented rear- end collisions by smoothly decreasing speed rather than executing harsh, late-stage stops.

* System Stability C Shutdown Protocol: Early iterations of the software suffered from "Actor Not Found" crashes upon termination. The implementation of carla.command.DestroyActor batching in the final shutdown sequence completely eliminated these crashes, ensuring safe and repeatable simulation lifecycles.

3. CHALLENGES C FUTURE SCOPE:

Challenges Faced:

* Synchronous Deadlock: Initial builds suffered from severe stuttering. When AI vehicles utilized set_autopilot(True) without syncing the Traffic Manager to the world's synchronous tick, micro-deadlocks occurred. This was resolved by forcing the Traffic Manager onto the same fixed delta clock as the main world.

* Ground Clutter C Phantom Braking: Radar signals bouncing off the road surface initially caused random AEB triggers. This was mitigated by strictly tuning the altitude geometry logic to ignore hits that did not correspond to physical, upright obstacles.

Future Scope (Phase 2 C 3):

* Vision-Based Perception: Upgrading from raw radar to camera-based Deep Learning (e.g., YOLOv8 or MobileNet) to classify objects. This would allow the system to apply different safety parameters for a semi-truck versus a bicycle.

* Multi-Sensor Fusion: Fusing LiDAR 3D point clouds with Camera 2D bounding boxes to create a comprehensive volumetric occupancy grid.

* Advanced Control Dynamics: Replacing the basic LKA proportional controller with a Stanley Controller or Model Predictive Control (MPC) to enable high-speed, stable cornering at highway speeds.

4. README.md:

Virtual Vahana: Level 2+ Advanced Driver Assistance System

[![CARLA Compatible](https://img.shields.io/badge/CARLA-0.9.x-blue.svg)](https://carla.org/)
[![Python 3.8+](https://img.shields.io/badge/Python-3.8%2B-brightgreen.svg)](https://www.python.org/)
[![License: MIT](https://img.shields.io/badge/License-MIT-yellow.svg)](https://opensource.org/licenses/MIT)

Project Overview

Virtual Vahana is an industry-grade, sensor-fused Advanced Driver Assistance System (ADAS) built within the CARLA Simulator. Designed specifically for unpredictable urban environments, the system adheres to a strict **"Human-First, Safety-Override"** philosophy.

While the human driver maintains primary control over the vehicle, the ADAS continuously runs spatial risk assessments in the background. It utilizes a 1D Kalman Filter to clean raw radar noise and a PID controller for longitudinal stability. If a critical collision vector involving another vehicle or a Vulnerable Road User (VRU/Pedestrian) is detected, the system executes an absolute manual override to safely stop the vehicle.

Key Features

* **Dynamic Adaptive Cruise Control (ACC):** Utilizes a PID controller and the kinematic "2.2-Second Rule" to maintain a speed-adaptive following distance.
* **Autonomous Emergency Braking (AEB):** Multi-stage braking logic (FCW Pre-charge -> AEB Full Override) based on dynamic Time-to-Collision (TTC) calculations.
* **Vulnerable Road User (VRU) Protection:** Widened radar FOV and geometric altitude filtering specifically calibrated to detect and protect pedestrians in crosswalks.
* **Lane Keeping Assist (LKA):** Proportional autosteer calculation based on waypoint cross-track error that gently assists the driver without locking the steering wheel.
* **Synchronous Simulation Engine:** Custom CARLA configuration synchronizing the World Tick, Traffic Manager, and Python Control Loop to prevent sensor desynchronization and physics engine stuttering at high speeds.
* **Futuristic HMI:** A Pygame-driven dashboard displaying real-time telemetry, radar target distance, dynamic safe gaps, and visual ADAS intervention alerts.

Prerequisites
To run this simulation, you will need the following installed on your machine:
* **CARLA Simulator** (v0.9.13 or newer recommended)
* **Python** (v3.8 or newer)
* **Python Libraries:**
 ```bash
 pip install pygame numpy

9.PROJECT OVERVIEW
Navigating unpredictable urban environments requires robust safety fallbacks. This project addresses the challenge of protecting Vulnerable Road Users (VRUs) such as pedestrians while maintaining manual driving autonomy.

Tested in CARLA's complex `Town10HD` map alongside 40 autonomous vehicles and 40 AI-driven pedestrians, this ADAS architecture seamlessly blends human inputs with overriding safety constraints using sensor-fused point-cloud data.

5. TEAM DETAILS:

Team Name:* Zerocollision
Institution:** Amrita Vishwa vidyapeetham
Track:** Virtual Vahana - Round 1
Team Members:**
 *[Member 1 Name] SivaKarthik B - **Team Lead & System Architect:** Oversees system integration, arbitration logic, and the synchronous CARLA simulation engine.
 *[Member 2 Name] Vishal K - **Perception Engineer (Radar):** Responsible for raw radar data extraction, geometric altitude filtering, and VRU (pedestrian) detection.
 *[Member 3 Name]Tharun M K - **Signal Processing Engineer:** Designs and tunes the 1D Kalman Filters for noise reduction in distance and relative velocity data.
 *[Member 4 Name] Periyannan - **Longitudinal Controls Engineer:** Develops the Adaptive Cruise Control (ACC) and tunes the PID controller for smooth acceleration/braking.
 *[Member 5 Name]Advaith- **Lateral Controls Engineer:** Calculates waypoint cross-track errors and implements the Lane Keeping Assist (LKA) proportional autosteer.
 *[Member 6 Name]Abhijith - **Safety Systems Engineer:** Formulates dynamic Time-to-Collision (TTC) calculations and programs the multi-stage AEB/FCW logic.
 *[Member 7 Name]Shashank- **HMI & UI Developer:** Builds the Pygame-
 ```

driven dashboard, rendering real-time telemetry, safe gaps, and visual intervention alerts.

*[Member 8 Name]Shyam - **Simulation Environment Specialist:** Manages the CARLA Traffic Manager, vehicle spawning, and pedestrian (walker) AI controllers.

*[Member 9 Name]Anirudh- **Kinematics & Validation Engineer:** Handles exponential moving average (EMA) actuator smoothing and conducts rigorous system testing/debugging.

10.STEPS TO RUN THE SIMULATOR:

Start the CARLA Server: Navigate to your CARLA installation folder and run the server executable.

- *Windows:* CarlaUE4.exe
- *Linux:* ./CarlaUE4.sh

(Optional but recommended for lower-end machines: Run with -quality-level=Low flag).

2. **Launch the ADAS Terminal:** Open a new command prompt or terminal, navigate to the directory containing this repository, and execute the main Python script:

Bash: - python VirtualVahanaADAS.py

3. **Initialization:** Please allow 3-5 seconds for the script to initialize synchronous mode, spawn the ego vehicle, and populate the urban environment with dynamic vehicle traffic and AI pedestrians.

Controls & Interface
The ego vehicle is driven manually via the keyboard. The system restricts maximum speed to the urban limit of **40 km/h**.

Key	Action	ADAS Interaction
W	Throttle	Modulated by ACC if approaching a target too quickly.
S	Brake	Overridden to 100% by AEB during critical TTC scenarios.
A / D	Steer Left / Right	Assisted by LKA; manual input easily overpowers the assist.
C	Toggle Camera	Cycles between 3rd Person, Dashcam, and Drone views.